CHAPTER 40

MICE and Iterative Imputation

Chained Equations, Multivariate Imputation, Convergence, Comparison

CampusX — 100 Days of Machine Learning
Feature Engineering Series, Videos 31–40

40.1 Why Univariate Imputation May Be Insufficient

Mean, median, and most-frequent imputation are **univariate** methods: they use only the information from a single feature to estimate missing values. Random sample imputation is also univariate. KNN imputation begins to use multivariate information but is limited to the k nearest neighbours.

In many real-world datasets, features are **correlated**. Income is correlated with education. Age is correlated with years of experience. Blood pressure is correlated with weight and age. When features are correlated, knowing the values of other features gives you information about the missing value. **Multivariate imputation** methods exploit these relationships.

40.2 MICE: Multivariate Imputation by Chained Equations

MICE (also called the chained equations approach or iterative imputation) was proposed by van Buuren, Boshuizen, and Knook (1999). The core idea is elegant: each feature with missing values is treated as the **target** of a regression model, using all other features as **predictors**.

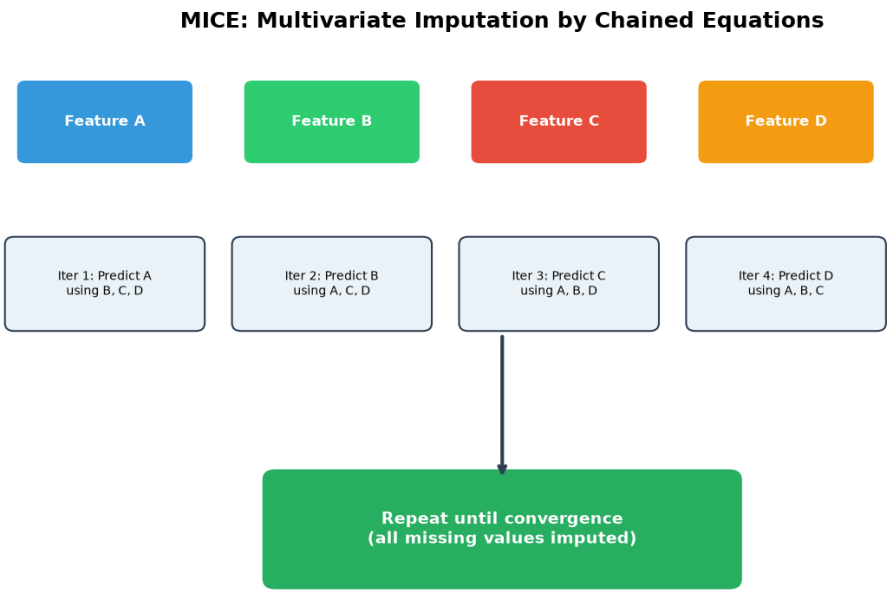


Figure 40.1: The MICE iterative process. Each feature with missing values is predicted using the other features. The process cycles through all features repeatedly until convergence.

40.3 The MICE Algorithm

The algorithm proceeds as follows:

1. **Initial imputation:** Fill all missing values with a simple method (e.g., mean or median) to create a complete initial dataset.
2. **For each feature j with missing values:**
   - Temporarily remove the observed values of feature j (treat them as missing).
   - Build a regression model: feature j is the target; all other features are predictors.
   - Predict the missing values of feature j using the model.
   - Replace the imputed values of feature j with these new predictions.

3. Cycle through all features with missing values. Repeat step 2 for each such feature. One full pass through all features is called one **iteration**.

4. Repeat iterations until the imputed values stabilise (converge). Convergence is typically assessed by monitoring the change in imputed values between iterations.

40.3.1 Mathematical Intuition

Suppose we have three features A, B, C, and feature A has missing values. In the first iteration, we fit a regression model:

$$A_{\text{pred}} = f(B, C) \tag{40.1}$$

and use it to predict the missing values of A. In the next step, if B also has missing values, we fit:

$$B_{\text{pred}} = g(A_{\text{new}}, C) \tag{40.2}$$

using the **updated** values of A (including the newly imputed ones). This sequential updating — where each feature's imputation uses the most recent estimates of other features — is what makes MICE "chained equations".

40.4 IterativeImputer in Scikit-learn

Scikit-learn implements MICE as `IterativeImputer` in `sklearn.impute`. As of Scikit-learn 1.x, it is considered experimental and requires importing from `sklearn.experimental`.

Parameter	Default	Description
estimator	BayesianRidge()	The regression model used to predict each feature
max_iter	10	Maximum number of imputation rounds
random_state	None	Controls randomness for reproducibility
initial_strategy	"mean"	Initial imputation: "mean", "median", "most_frequent"
imputation_order	"ascending"	Order in which features are imputed: "ascending", "descending", "arabic", "random"
sample_posterior	False	If True, uses sampling from posterior; if False, uses point estimates
tol	1e-3	Convergence tolerance: stop when change < tol
n_nearest_features	None	Number of features to use per estimator (for speed)

Table 40.1: IterativeImputer parameters.

```
import numpy as np
import pandas as pd
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.linear_model import BayesianRidge

# Dataset with correlated features and missing values
np.random.seed(42)
n = 300

# Create correlated features
education = np.random.normal(14, 3, n) # Years of education
income = 30000 + 5000 * education + np.random.normal(0, 15000, n)
experience = np.random.normal(10, 5, n)
age = 22 + experience + np.random.normal(0, 3, n)
credit_score = 600 + 10 * education + 2 * income/1000 + np.random.normal(0, 40, n)

df = pd.DataFrame({
'education': education,
'income': income,
'experience': experience,
'age': age,
'credit_score': credit_score
})

# Introduce missing values (MAR: less educated more likely to skip income)
for i in range(n):
if education[i] < 12 and np.random.random() < 0.4:
df.loc[i, 'income'] = np.nan
if np.random.random() < 0.1:
df.loc[i, 'credit_score'] = np.nan
if np.random.random() < 0.08:
df.loc[i, 'experience'] = np.nan
```

```
print("Missing per column:")
print(df.isnull().sum())

# Apply MICE imputation
imputer = IterativeImputer(
estimator=BayesianRidge(),
max_iter=10,
random_state=42,
sample_posterior=True
)
df_imputed = pd.DataFrame(
imputer.fit_transform(df),
columns=df.columns
)

print(f"\nShape before: {df.shape}, after: {df_imputed.shape}")
print(f"Missing after: {df_imputed.isnull().sum().sum()}")

# Compare original and imputed correlations
print("\n=== CORRELATION COMPARISON ===")
print("Original (observed pairs):")
print(df.corr().round(3))
print("\nAfter MICE imputation:")
print(df_imputed.corr().round(3))
```

Listing 40.1: MICE imputation with IterativeImputer.

40.5 Comparison: KNN vs MICE

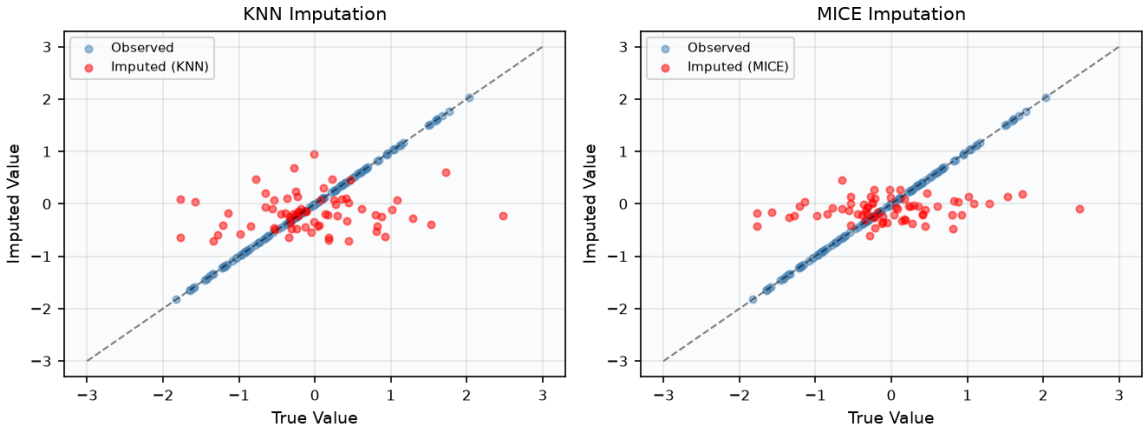


Figure 40.2: KNN vs MICE imputation on correlated data. Both methods recover the relationship between true and imputed values, but MICE tends to produce slightly better-calibrated estimates when features are strongly correlated.

Aspect	KNNImputer	IterativeImputer (MICE)
Approach	Find k similar observations, average their values	Fit regression model per feature, predict missing values
Feature relationships	Implicitly via distance metric	Explicitly via regression coefficients
Computation	$O(n^2 \times p)$ — expensive for large n	$O(\text{iterations} \times n \times p^2)$ — depends on max_iter
Scalability	Poor for large datasets	Better for large datasets
Hyperparameters	n_neighbors, weights	estimator, max_iter, imputation_order
Distribution preservation	Good (samples from observed values)	Good (regression predictions vary)
Best for	Small-to-medium datasets with moderate correlation	Datasets with strong feature correlations

Table 40.2: KNNImputer vs IterativeImputer comparison.

40.6 Convergence and Stopping Criteria

IterativeImputer stops when one of two conditions is met:

1. The maximum number of iterations (`max_iter`) is reached.
2. The change in imputed values between consecutive iterations falls below `tol` (default 1e-3).

In practice, 5–10 iterations are usually sufficient for convergence. If the imputed values are still changing significantly at max_iter, increase the iteration count or check whether the regression model is appropriate for the feature being imputed.

40.7 Advantages and Assumptions

40.7.1 Advantages

• **Uses all features:** Each feature's imputation leverages information from all other features.

• **Preserves correlations:** The iterative process maintains the multivariate relationships in the data.

• **Flexible estimator:** Any regression model can be used (BayesianRidge, DecisionTree, KNeighbors, etc.).

• **Handles different types:** Different estimators can be used for different feature types.

40.7.2 Assumptions

• **MAR assumption:** MICE assumes data is Missing at Random (missingness depends on observed data, not the missing values themselves).

• **Model specification:** The quality of imputation depends on whether the chosen regression model correctly captures the relationship between features.

• **Convergence:** The algorithm assumes the iterative process converges. For highly complex relationships, convergence may be slow or unstable.

When NOT to Use MICE

Avoid MICE when: (1) the dataset is very large and computation time is critical; (2) features are essentially univariate (no correlations to exploit); (3) the missingness is MNAR (MICE assumes MAR); (4) the relationship between features is highly non-linear and the default linear estimator cannot capture it. In such cases, use a tree-based estimator within IterativeImputer or fall back to simpler methods.

40.8 Key Takeaways

★ MICE treats each feature with missing values as a regression target, using all other features as predictors. The process iterates until convergence, preserving multivariate relationships. Scikit-learn implements this as IterativeImputer. It is more powerful than KNN for correlated data but assumes MAR and requires a suitable regression model.

40.9 Interview Questions

1. Explain the MICE algorithm step by step.

2. How does IterativeImputer differ from KNNImputer?
3. What is the role of the estimator parameter in IterativeImputer?
4. What does the max_iter parameter control, and what is a typical value?
5. What assumption does MICE make about the missingness mechanism?
6. How would you handle a mix of numerical and categorical features with MICE?